

Requirements Specification for a Knowledge Graph Exploration Application

Version 1.2
6 October 2026
Literature cutoff: 29 August 2026

Abstract

This specification defines a domain-independent application for researchers and users without graph-query expertise. Version 1 supports both Resource Description Framework (RDF) graphs queried through SPARQL and property graphs queried through Cypher. Users search for entities, refine results with facets, navigate relationships, find paths, construct queries with guidance, switch views, and save findings with their source and query context. Requirements are linked to primary literature and paired acceptance scenarios. Numerical thresholds are proposed engineering targets, not measured results for the application.

Contents

1	Purpose, scope, and success criteria	3
1.1	Priority conventions	3
2	Evidence basis and design rationale	3
3	Users and tasks	4
4	Graph models, connections, and data semantics	5
4.1	Version 1 visual query profile	5
4.2	Dialect compatibility declaration	5
5	Functional requirements	6
5.1	Connections and graph fidelity	6
5.2	Search, schema inspection, and facets	6
5.3	Navigation and relationship discovery	7
5.4	Visual and native query construction	7
5.5	Explainable guidance	8
5.6	Coordinated views and complexity control	8
5.7	Workspace, provenance, and export	9
5.8	Recovery and evaluation support	9
6	Quality requirements and operational constraints	10

7	Acceptance and evaluation plan	11
7.1	Paired fixtures	11
7.2	Visual query coverage for AC-04	12
7.3	Benchmark protocol	13
7.4	Task-based usability evaluation	13
8	Traceability and release criteria	13
A	Local primary-source identifiers	15
B	Glossary	15

1 Purpose, scope, and success criteria

The application shall let users revise search terms, filters, and relationship constraints while inspecting graph data. The core tasks are to find entities, inspect connections, restrict a result set, formulate a query, and save a finding with its supporting data. Researchers shall inspect native queries and source values. Non-experts shall complete the core workflow without writing SPARQL or Cypher.

Version 1 requires RDF/SPARQL and property-graph/Cypher connectors for every mandatory exploration workflow and the visual query profile in Section 4.1.

The application shall operate independently of a domain vocabulary. Source profiling may inspect labels, types, relationships, and value types. Users may configure presentation labels and preferred facets without changing application code. The application explores external graph sources.

Source connections are read-only. Version 1 excludes graph construction and editing, ontology alignment, bulk ingestion into an application-owned store, autonomous external actions, cross-source query federation, arbitrary language conversion, and persistent collaboration services. Users may edit workspace annotations and saved queries locally.

1.1 Priority conventions

Must requirements are release conditions. **Should** requirements improve the release but cannot compensate for a missing Must capability. Functional requirements (FR), non-functional requirements (NFR), and acceptance scenarios (AC) use stable identifiers; a changed requirement shall receive a new version and updated tests. The requirements specify behaviour; implementation technology is left open.

2 Evidence basis and design rationale

The literature cutoff is 29 August 2026. Table 1 lists eight research papers and four technical specifications, with section references. The papers inform the proposed features. Mandatory support for both graph models, priorities, and numerical thresholds are project decisions.

Each origin statement distinguishes *documented precedent* (behaviour described in a paper), *normative basis* (adopted language or accessibility semantics), *design extension* (features added to the precedent), and *project decision* (scope, policy, formats, targets, or tests). A documented feature alone does not establish a usability benefit. Citations support the stated precedent or semantics.

Tabulator combines exploration and analysis with source inspection (Berners-Lee et al., 2006, Sections 3.1–3.4). /facet describes dynamic facets, search, and cross-type constraints (Hildebrand et al., 2006, Sections 3–4); RelFinder addresses disambiguation and relationship search (Heim et al., 2009, Sections 2.1–2.3). Sparklis combines stepwise query refinement with controlled-language presentation (Ferré, 2017, Sections 3 and 5).

LODmilla saves graph state and reverses exploration actions (Micsik et al., 2014, Section 3.1). Sampo-UI connects a faceted result set with analytical perspectives (Ikkala et al., 2022, Sections 3.1–3.2 and 4.4–4.5). Ranked graph-query expansions motivate inspectable guidance (Lissandrini et al., 2020, Sections 3–4). SemSpect explicitly addresses RDF and labelled property graphs and uses aggregation to reduce visual clutter (Liebig et al., 2023, Abstract and Sections 1–2).

Table 1: Primary evidence register. Locators refer to named or numbered source sections.

ID	Primary source, locator, and relevant observation
E01	Tabulator (Berners-Lee et al., 2006), Sections 3.1–3.4: source provenance and network status; exploration and analysis; outline; query-by-example and alternative result views.
E02	/facet (Hildebrand et al., 2006), Sections 3.1–3.5 and 4.1–4.5: dynamic facets, semantic keyword search, multi-type queries, runtime facet specification, and facet-specific displays.
E03	RelFinder (Heim et al., 2009), Sections 2.1–2.3: entity disambiguation, iterative relationship search, labelled directed edges, and graph display control.
E04	Sparklis (Ferré, 2017), Section 3, <i>Principles and architecture</i> ; Section 5, <i>Capabilities and limitations</i> ; Section 5.5, <i>Portability: No required configuration</i> : guided refinement, readable queries, and endpoint-driven vocabulary discovery.
E05	LODmilla (Micsik et al., 2014), Section 3.1, <i>Frontend</i> , subheadings <i>Saving/Sharing/Loading</i> and <i>Undo</i> ; Section 3.2, <i>Backend</i> : preserved graph state, undo, source traversal, and bounded search.
E06	Sampo-UI (Ikkala et al., 2022), Sections 3.1–3.2 and 4.1, 4.4–4.5: facets, shared result perspectives, interface structure, and CSV query-result export.
E07	Graph-query suggestions (Lissandrini et al., 2020), Section 3, <i>Problem Formulation</i> ; Section 4, <i>Graph-Query Suggestion Model</i> : exemplar queries and ranked edge expansions.
E08	SemSpect (Liebig et al., 2023), Abstract; Section 1, <i>Motivation</i> ; Section 2, <i>Querying and Visual Exploration Beyond the Hairball</i> : both graph families, aggregation, and overview-to-detail exploration.
E09	RDF 1.1 Concepts (World Wide Web Consortium, 2014), Sections 3.1–3.4 and 4: term identity, literal structure, blank-node scope, and datasets.
E10	SPARQL 1.1 Query Language (Harris & Seaborne, 2013), Sections 5–9 and 11–17: patterns, optionality, alternatives, paths, aggregates, result modifiers, and value operations. The selected visual subset is a project decision.
E11	openCypher (openCypher, 2026), pinned release 2024.3 grammar and TCK: native-language semantics and scenarios with graphs, parameters, expected results or errors.
E12	WCAG 2.2 (World Wide Web Consortium, 2024), Success Criteria 1.1.1, 2.1.1, 2.4.7, 4.1.2 and Section 5.2.1: text alternatives, keyboard access, visible focus, control semantics, and conformance levels.

Table 2: Design considerations and sources.

ID	Sources	Design consideration
G01	E01, E02, E04	Vocabulary and query-language knowledge can obstruct entry. Provide readable search, facets, and progressive query construction.
G02	E03, E05, E08	Expansion increases visual density and can lose context. Bound displays, expose paths, and preserve recovery.
G03	E04, E07	Guidance changes exploration direction. Explain its basis and distinguish verified results from estimates.
G04	E01, E05	Findings require source, query, and session context to be revisited. Preserve context in saves and exports.
G05	E06, E08	Set inspection and aggregation require coordinated views beyond a node-link display.
G06	E01, E04, E08	Source access, graph semantics, and limits differ. Test both connectors and their failure modes separately.

3 Users and tasks

Researchers explore unfamiliar data, formulate and inspect queries, compare refinements, check values and scope, and export reproducible evidence. **Non-experts** start from a phrase or entity,

disambiguate labels, follow relationships, refine sets, and preserve findings through readable controls. Either user may select a configured connection or enter an authorised source profile. A profile contains graph family, address, dataset or graph scope, authentication reference, and query dialect. It does not imply permission to modify the source.

The core sequence is: select source; search or choose a type; inspect entities; refine facets; expand relationships or find a path; build or inspect a query; change view; save a finding; export its supporting context. Users may repeat or branch these steps without completing a fixed wizard.

4 Graph models, connections, and data semantics

RDF terms and dataset context follow the RDF 1.1 data model ([World Wide Web Consortium, 2014](#)). Resources retain their Internationalized Resource Identifier (IRI) or source-scoped blank-node identity. Literals retain lexical value, datatype, and language tag. Named-graph context remains inspectable when provided. Blank-node labels shall not be treated as persistent cross-session identifiers.

Property-graph nodes retain source-scoped identifiers and all labels. Relationships retain identifiers where supplied, type, direction, endpoints, and properties. Parallel relationships and self-loops remain distinguishable, including when grouped for display.

The shared interface shall not erase model-specific semantics. Missing properties, unbound variables, empty strings, zero, and false remain distinct. Observed schema information is labelled as observed; declared schema retains its source. Source-side read permissions and application operation controls enforce read-only access. SPARQL Update, mutating Cypher statements, and administrative or mutating procedures are rejected.

4.1 Version 1 visual query profile

The builder supports connected patterns, variables, repeated-variable joins, type or label constraints, directed and inverse relationships, optional patterns, alternatives, equality and inequality, string matching, numeric and temporal ranges, projection, distinct results, ordering, limits, and grouped COUNT, SUM, AVG, MIN, and MAX. Filters and aggregates check applicable value types.

The RDF connector generates SPARQL 1.1 read queries ([Harris & Seaborne, 2013](#)). The property-graph connector generates the supported operations in its declared Cypher dialect. Each connector declares its language and version and retains its model-specific semantics. Native read queries may use additional supported constructs. Queries outside the visual profile remain in native mode; unsupported constructs are not discarded or assigned an invented visual translation. Changing graph family requires a fresh query or explicit supported reconstruction.

4.2 Dialect compatibility declaration

Each Cypher connector profile shall identify its implementation dialect and version, its language-reference release and commit, and semantic differences relevant to the mandatory visual query profile. The openCypher specification, grammar, and Technology Compatibility Kit (TCK) provide a versioned reference ([openCypher, 2026](#)). The inspected reference is release 2024.3, published on 20 March 2026, at commit 677cbafabb8c3c5eed458fd3b1ec0daec8d67d23.

The compatibility manifest shall map every mandatory visual construct to supported native syntax, expected semantics, and its acceptance cases. A connector may implement another declared Cypher dialect; every Must workflow and operator must still pass separately on both graph families. Unsupported mandatory constructs prevent release. Relevant read-query TCK scenarios may

inform language checks, but a selected subset does not establish complete TCK conformance. This declaration does not prescribe a database product or require arbitrary SPARQL-to-Cypher translation.

5 Functional requirements

5.1 Connections and graph fidelity

FR-01 — Read-only dual-backend connections

Priority: Must. **Tests:** AC-01, AC-08.

Connect to an RDF/SPARQL endpoint and a property-graph/Cypher source through profiles. Expose source scope, dialect, connection, and authentication status. Changing source preserves the earlier workspace as a separate context. Both connectors implement every mandatory workflow.

Origin (Documented precedent; project decision). Sparklis, E04, Section 5.5 (Ferré, 2017), describes SPARQL endpoint access; SemSpect, E08, Section 2 (Liebig et al., 2023), supports RDF and labelled property graphs. **Derivation.** Mandatory coverage of both models in version 1, connection profiles, read-only access, authentication feedback, and separate source workspaces are project decisions. SemSpect’s indexed RDF dumps do not establish remote endpoint support.

FR-02 — Identity and value preservation

Priority: Must. **Tests:** AC-01, AC-07.

Details, table cells, and exports preserve the semantics above. Equal labels never merge unequal entities. Identifiers remain inspectable. Language preference changes presentation, not source values.

Origin (Normative basis; design extension). RDF Concepts, E09, Sections 3.1–3.4 and 4 (World Wide Web Consortium, 2014), defines terms and dataset identity; the declared Cypher reference, E11 (openCypher, 2026), governs property-graph semantics. **Derivation.** Preserving those distinctions in every view and export, and preventing label-based entity merging, are interface obligations derived from the adopted semantics. Language preference remains a presentation choice.

5.2 Search, schema inspection, and facets

FR-03 — Entity search and disambiguation

Priority: Must. **Tests:** AC-02.

Search by identifier or label text. Candidates expose type or label, source, and identifier. State search scope and result limits. Expose discovered entity types or labels, relationships, and property value types.

Origin (Documented precedent; design extension). /facet, E02, Section 4.2 (Hildebrand et al., 2006), describes keyword search; RelFinder, E03, Section 2.1 (Heim et al., 2009), describes entity disambiguation; Sparklis, E04, Section 5.5 (Ferré, 2017), discovers endpoint vocabulary. **Derivation.** The application adds candidate identifiers, source and type information, search bounds, and property-value inspection.

FR-04 — Faceted refinement

Priority: Must. **Tests:** AC-02.

Combine categorical, numeric, and temporal facets. Selections within a categorical facet use OR; selections across facets use AND. Show removable constraints and exact or estimated counts. Empty results preserve constraints and support removal or undo. Missing values remain distinct from empty strings.

Origin (Documented precedent; design extension; project decision). /facet, E02, Sections 3–4 (Hildebrand et al., 2006), supplies dynamic facets; Sampo-UI, E06, Section 4.4 (Ikkala et al., 2022), supplies active filters, disjunctive choices and date/integer ranges. **Derivation.** OR within and AND across facets is the adopted application contract. Estimate labels, empty-result recovery and missing-value distinctions specify how that contract is exposed.

5.3 Navigation and relationship discovery

FR-05 — Incremental neighbourhood navigation

Priority: Must. **Tests:** AC-03.

Inspect incoming and outgoing relationships; expand a relationship type; pin an entity; collapse and restore expansions. Expansion is bounded and pageable. Collapse hides view elements while preserving source data and exploration history. Edges retain direction and type.

Origin (Documented precedent; design extension). Tabulator, E01, Section 3.3 (Berners-Lee et al., 2006), exposes incoming/outgoing links; RelFinder, E03, Section 2.3 (Heim et al., 2009), supports directed labelled graph inspection and pinning; LODmilla, E05, Section 3.1 (Micsik et al., 2014), opens related resources and removes canvas nodes. **Derivation.** Pageable expansion, restoration after collapse, and preservation of source and history state define the proposed workspace behaviour.

FR-06 — Bounded relationship paths

Priority: Must. **Tests:** AC-03.

Search between two selected entities with relationship and direction constraints. Default depth is three hops, configurable up to six. Initially show at most ten paths ordered by hop count and a deterministic tie-break. Expose path identities, directions, and source scope. A reached limit is not proof that no path exists.

Origin (Documented precedent; design extension; project decision). RelFinder, E03, Section 2.2 (Heim et al., 2009), searches paths with configurable maximum length; LODmilla, E05, Section 3.2 (Micsik et al., 2014), bounds traversal and results. **Derivation.** Three default hops, six maximum hops, ten initial paths and deterministic ordering are selected project limits. The application adds direction controls and messages that distinguish a search limit from a missing path.

5.4 Visual and native query construction

FR-07 — Visual query profile

Priority: Must. **Tests:** AC-04.

Implement Section 4.1 for both models. Add, change, and remove constraints incrementally. Show generated query text, parameters, scope, and a readable description before execution.

Origin (Documented precedent; normative basis; project decision). Tabulator, E01, Section 3.4 (Berners-Lee et al., 2006), describes query formulation; Sparklis, E04, Sections 3 and 5.1 (Ferré, 2017), supports incremental guided construction. E10 and E11 provide native-language semantics (Harris & Seaborne, 2013; openCypher, 2026). **Derivation.** The exact operator profile and mandatory coverage on both models are project decisions. Showing query text, parameters, scope and explanation before execution is the proposed interface contract.

FR-08 — Expert read-query mode

Priority: Must. **Tests:** AC-04, AC-08.

Inspect, edit, execute, and save native read queries with language and dialect. Native editing does not overwrite the saved visual state silently. Errors identify rejected syntax or capabilities. Queries outside the visual profile remain usable in native mode.

Origin (Documented precedent; design extension; project decision). Tabulator, E01, Section 3.4 (Berners-Lee et al., 2006), permits viewing and editing SPARQL query text. **Derivation.** Separate native and visual state, saved native queries, capability-specific errors and retention of constructs outside the visual subset extend that precedent. Native Cypher mode and read-only enforcement are scope decisions; the paper does not establish arbitrary cross-language translation.

5.5 Explainable guidance

FR-09 — Inspectable next-step suggestions

Priority: Must. **Tests:** AC-05.

Provide user-requested eligible relationships, refinements, or result views. Each suggestion states its basis: schema compatibility, checked counts, or ranking criteria. Distinguish schema-valid, estimated, and verified non-empty suggestions. Users may ignore guidance.

Origin (Documented precedent; design extension). Sparklis, E04, Sections 3 and 5.2 (Ferré, 2017), supplies data-driven refinements; Graph-Query Suggestions, E07, Sections 3–4 (Lissandrini et al., 2020), ranks graph-query expansions. **Derivation.** The application explains each suggestion’s basis and distinguishes schema compatibility, estimates and verified non-empty results. Result-view suggestions extend the cited query-expansion mechanisms.

FR-10 — Readable query descriptions

Priority: Must. **Tests:** AC-05.

Describe active query and filters in plain English from the current query state. Distinguish conjunction, alternatives, optional values, aggregates, and limits. Descriptions use source labels and do not imply stronger findings than the query produces.

Origin (Documented precedent; design extension; project decision). Sparklis, E04, Section 5.3 (Ferré, 2017), verbalizes query state with source labels in English or French. **Derivation.** Selecting English is a project decision. Faithful descriptions of every selected operator, its scope and limits are correctness obligations derived for this application.

5.6 Coordinated views and complexity control

FR-11 — Graph, table, and detail coordination

Priority: Must. **Tests:** AC-06.

Views use the same result scope and preserve selection, filters, and query context. A selected entity is identifiable across views. Textual controls provide equivalent core inspection and navigation when the graph cannot be used.

Origin (Documented precedent; normative basis; design extension). Tabulator, E01, Section 3.4 (Berners-Lee et al., 2006), presents alternative query-result views; Sampo-UI, E06, Sections 4.4–4.5 (Ikkala et al., 2022), coordinates facet and result state. E12 grounds accessible interaction (World Wide Web Consortium, 2024). **Derivation.** Preserving entity selection across views and offering equivalent textual operation are application-level coordination and accessibility adaptations.

FR-12 — Aggregation and display limits

Priority: Must. **Tests:** AC-06, AC-09.

Show counts and grouped summaries with computation scope and completeness. Drill from an aggregate to its entities. Initially display at most 500 nodes and 1,000 relationships; use grouping, paging, or selected slices beyond this envelope. Display omission never changes the reported query result.

Origin (Documented precedent; design extension; project decision). SemSpect, E08, Sections 1–2 (Liebig et al., 2023), uses aggregation and overview/detail; Sampo-UI, E06, Sections 3.1 and 4.5 (Ikkala et al., 2022), provides summaries, paging and clustering. **Derivation.** The display limits of 500 nodes and 1,000 relationships are project targets. Completeness labels, aggregation scope and query totals unchanged by display limits are application extensions.

FR-12a (Should). Offer temporal and geographical views when source values support them without inventing dates, coordinates, or units. Otherwise retain table inspection.

Origin (Documented precedent; design extension; project decision). Tabulator, E01, Section 3.4 (Berners-Lee et al., 2006), includes map/calendar views; /facet, E02, Section 4.5 (Hildebrand et al., 2006), includes a timeline; Sampo-UI, E06, Section 4.5 (Ikkala et al., 2022), includes geographic and temporal perspectives. **Derivation.** Should priority, value-availability checks, preservation of units and the table fallback are choices for this domain-independent application.

5.7 Workspace, provenance, and export

FR-13 — Recoverable exploration workspace

Priority: Must. **Tests:** AC-07.

Save and reopen queries, filters, selections, annotations, positions, source references, and steps. Undo and redo restore view and query state. Branching preserves earlier branches. Reopening distinguishes saved observations from refreshed results.

Origin (Documented precedent; design extension). LODmilla, E05, Section 3.1, Saving/Sharing/Loading and Undo (Micsik et al., 2014), preserves graph state and reverses an action; Sparklis, E04, Section 4 (Ferré, 2017), provides query permalinks. **Derivation.** Redo, branching, annotations, additional workspace fields, and the distinction between saved and refreshed results extend the cited capabilities.

FR-14 — Data and query export

Priority: Must. **Tests:** AC-07.

Export tables as comma-separated values (CSV) and typed JavaScript Object Notation (JSON), native query text, and graph views as Scalable Vector Graphics (SVG). RDF statement exports retain graph context; property-graph exports retain identities and properties. Indicate full versus bounded results. Typed JSON preserves distinctions flattened for CSV presentation.

Origin (Documented precedent; normative basis; project decision). Sampo-UI, E06, Section 4.1 (Ikkala et al., 2022), exports CSV query results; SemSpect, E08, Section 2 (Liebig et al., 2023), exports SPARQL/Cypher queries. E09 and E11 ground model fidelity (World Wide Web Consortium, 2014; openCypher, 2026). **Derivation.** CSV, typed JSON and SVG are selected output formats. Preserving typed identities and graph context, and disclosing bounds, are derived export obligations.

FR-15 — Finding provenance

Priority: Must. **Tests:** AC-07.

An evidence bundle includes finding, source profile identifier, graph scope, retrieval time, native query and parameters, filters, limits, available source version, and annotations. An unavailable version is recorded as unknown. Provenance does not imply independent verification of returned assertions.

Origin (Documented precedent; design extension). Tabulator, E01, Section 3.1 (Berners-Lee et al., 2006), exposes source provenance; LODmilla, E05, Section 3.1 (Micsik et al., 2014), saves graph state representing findings. **Derivation.** The source, query, time and version fields in the evidence bundle extend the cited features.

5.8 Recovery and evaluation support

FR-16 — Cancellation and recoverable failure

Priority: Must. **Tests:** AC-08.

Cancel or retry requests without losing the last valid workspace. Distinguish connection, authentication, query, timeout, and decoding failures. Late cancelled results do not replace current state. Retained results show timestamp and stale status.

Origin (Documented precedent; design extension). Tabulator, E01, Section 3.1 (Berners-Lee et al., 2006), exposes network progress and failures; Sparklis, E04, Sections 5.4–5.5 (Ferré, 2017), describes endpoint-dependent performance and limits. **Derivation.** Cancellation, retry, differentiated failures, retained stale results and suppression of late responses define this application’s recovery contract.

FR-17 (Should). Optional evaluation recording captures task identifiers, enabled interaction events, completion, time, and errors. It excludes credentials and unnecessary data values. Users inspect, export, and delete records. Recording is disabled by default.

Origin (Documented precedent; project decision). Sparklis, E04, Section 6 introductory paragraphs (Ferré, 2017), describes navigation logs and a user logging switch. **Derivation.** Task/error instrumentation, inspect/export/delete controls, data minimization and recording disabled by default are project decisions. The precedent’s IP/query/endpoint logging does not supply these privacy defaults.

6 Quality requirements and operational constraints

The timings below are engineering targets under the declared benchmark conditions. Measure local user interface (UI) feedback separately from source-query execution.

Table 3: Mandatory quality requirements.

ID	Requirement and test
NFR-01	Exact queries match the fixture oracle for every mandatory case. Preserve identity, direction, datatype, named scope, and aggregate scope. Test both backend families separately. Origin: normative basis and project verification. E09–E11 (World Wide Web Consortium, 2014; Harris & Seaborne, 2013; openCypher, 2026) define the data and query semantics. Fixture oracles and separate paired acceptance are the verification method selected here.
NFR-02	Core workflows support keyboard use, visible focus, meaningful control names, and a textual graph alternative. Target Web Content Accessibility Guidelines (WCAG) 2.2 level AA (World Wide Web Consortium, 2024). Use manual and automated checks; automated scanning alone is insufficient. Origin: normative basis and project adaptation. E12, WCAG 2.2 SC 1.1.1, 2.1.1, 2.4.7, 4.1.2 and Section 5.2.1 (World Wide Web Consortium, 2024). The AA target, textual workflow and combined manual/automated checks are this project’s adoption and test protocol.
NFR-03	Proposed local selection, view-switching, and undo feedback target: 95th percentile of 200 ms within the display limits in FR-12 on the declared benchmark. Measure network requests separately. Origin: project performance target. SemSpect, E08, Sections 1–2 (Liebig et al., 2023), motivates complexity control. The percentile, 200-ms value, operations and display limits are selected engineering targets, not paper-derived thresholds.
NFR-04	Proposed search, facet, and single-hop expansion target: 95th percentile of 2 s end-to-end on the controlled warm-source benchmark. Report cold runs, bounded paths, and public-endpoint variability separately. Arbitrary queries and external sources have no universal latency guarantee. Origin: documented motivation and project target. Sparklis, E04, Section 5.4 (Ferré, 2017), explains endpoint-dependent response times. The 2-s percentile target and controlled warm-source conditions are project decisions.
NFR-05	Default timeout is 10 s. Initial expansion retrieves at most 100 relationships; table pages contain at most 100 rows. Cancellation updates local state within 200 ms. Where source cancellation is unavailable, discard late results and report outstanding source execution accurately. Origin: documented precedent and project limits. Sparklis, E04, Section 5.5 (Ferré, 2017), and LODmilla, E05, Section 3.2 (Micsik et al., 2014), motivate bounded work. All stated timeout, count and cancellation values are selected here; late-result handling is a design extension.
NFR-06	Keep credentials separate from workspace data. Never expose them in exports, URLs, histories, or logs. Stored credentials require a protected mechanism. Source permissions and operation controls enforce read-only access. Origin: project security and privacy policy. Credential storage, export and logging controls are project security decisions under the read-only scope in Section 1.
NFR-07	Inject timeout, disconnect, malformed result, and expired authentication. The last valid workspace survives with query, source, constraints, completeness, and stale state. Origin: design extension and project test policy. Tabulator, E01, Section 3.1 (Berners-Lee et al., 2006), shows network failures; LODmilla, E05, Section 3.1 (Micsik et al., 2014), saves graph state. The injected failure set and retained-state pass condition are specified here.

ID	Requirement and test
NFR-08	Run mandatory tasks on at least two distinct domain vocabularies per backend without application-code changes. Presentation configuration may differ. Origin: documented motivation and project portability test. Sparklis, E04, Section 5.5 (Ferré, 2017), discovers endpoint structure; /facet, E02, Sections 3.4 and 4.4 (Hildebrand et al., 2006), configures facets at runtime. Two vocabularies per backend and the no-code-change criterion are project decisions.
NFR-09	Record fixture version, expected results, application and connector versions, dialect, language-reference release and commit, compatibility manifest, environment, cache conditions, failures, and timings for each acceptance run. Report backend results separately. Origin: reference-based verification and project reporting policy. E11, the pinned openCypher TCK README (openCypher, 2026), describes scenario inputs and expected results/errors. The acceptance-run manifest, cache/timing metadata and separate backend reports extend that reference for this application.

7 Acceptance and evaluation plan

7.1 Paired fixtures

Provide RDF and property-graph representations of equivalent task facts, plus separate model-specific cases. Include ambiguous labels, disconnected entities, directed and inverse relationships, cycles, missing properties, empty strings, zero, false, temporal values, and known paths. RDF tests add typed and language-tagged literals, blank nodes, and named graphs. Property-graph tests add multiple labels, relationship properties, parallel relationships, and self-loops.

The paired fixtures, result oracles and pass conditions are project verification choices. Each scenario below lists the requirements it tests. Every task has expected identities, values, scope, and completeness. Compare task-level answers through a declared mapping without erasing model-specific distinctions. Equivalent tasks need not use identical query text or storage representation. Execute every scenario below on both connectors.

Table 4: Mandatory paired acceptance scenarios.

ID	Scenario and pass condition
AC-01	Connect, choose scope, inspect equal-labelled entities and their incoming/outgoing edges, and attempt mutation. Identities and model-specific values are retained; entities remain separate; reads succeed and writes are rejected. Named graph and relationship-property cases pass independently. Origin: derived acceptance test for FR-01, FR-02, NFR-01 and NFR-06.
AC-02	Search, disambiguate, combine categorical and range facets, remove a filter, and select missing values. Results match the oracle; OR/AND semantics, counts, scope restoration, completeness, and missing-value distinctions are correct. Origin: derived acceptance test for FR-03 and FR-04.
AC-03	Expand, collapse, restore, and search a known path under direction/type/hop constraints. Neighbourhoods and paths match the oracle. Limits and omissions are visible; no-path and reached-limit outcomes remain distinct; undo preserves context. Origin: derived acceptance test for FR-05 and FR-06.

ID	Scenario and pass condition
AC-04	Exercise every operator in Section 4.1 using the coverage cases below, independently on RDF/SPARQL and property-graph/Cypher. Inspect generated native text and edit a separate native query. Results and errors match the oracle; descriptions match semantics; native edits preserve visual state; unsupported translation is not fabricated. A compound example does not replace operator-level coverage. Origin: derived acceptance test for FR-07, FR-08 and NFR-01.
AC-05	Request and inspect guidance, accept or reject it, create an empty result, and remove a constraint. Suggestion certainty is explicit; chosen actions update query and description consistently; guidance remains optional; previous state survives. Origin: derived acceptance test for FR-09 and FR-10.
AC-06	Select an entity across views, filter, group results, and drill into an aggregate. Scope and selection remain consistent; counts reconcile with underlying exact or bounded data; textual controls support equivalent inspection. Origin: derived acceptance test for FR-11, FR-12 and NFR-02.
AC-07	Save/reopen a branch, replay after fixture change, and export results, query, view, and evidence bundle. State survives; change is flagged; identity, typed values, source, query, times, limits, and completeness remain correct; credentials are absent. Origin: derived acceptance test for FR-13, FR-14, FR-15 and NFR-06.
AC-08	Inject timeout, disconnect, authentication and decoding errors, and late cancelled results. Errors remain distinct; valid state survives with stale indication; retry succeeds; late results cannot overwrite state. Origin: derived acceptance test for FR-16, NFR-05 and NFR-07.
AC-09	Run display and performance workloads, exceed the display cap, and repeat on a second vocabulary. Grouping and limits work; declared benchmark targets are met; both backends and domains pass independently. Origin: derived acceptance test for FR-12, NFR-03, NFR-04 and NFR-08.

7.2 Visual query coverage for AC-04

Every operator below requires individual and applicable combined cases on both connectors. Fixtures declare expected identities, typed values, multiplicities, ordering, and completeness for each case. Include positive cases, empty-result cases, and validation cases for inapplicable value types. Each of the five aggregate functions is tested separately. Passing one aggregate or one backend does not satisfy the visual profile.

Table 5: Mandatory operator coverage within AC-04.

Operators	Required checks on RDF/SPARQL and property-graph/Cypher
Connected patterns, variables, repeated-variable joins, type/label constraints	Build and edit each construct; verify shared-variable equality, join scope, type or label membership, and returned identities against the oracle. Equal labels do not substitute for equal entities.
Directed and inverse relationships	Query each direction separately and in joined patterns. Verify relation type, endpoints, and the expected effect of reversing direction.
Optional patterns and alternatives	Check a matched and unmatched optional branch, each alternative, and their combinations. Verify missing or unbound values, branch scope, and result multiplicity.
Equality, inequality, string matching, numeric ranges, temporal ranges	Test each operator separately and in combined filters, including range boundaries. Verify datatype-appropriate comparison and distinctions between missing values, empty strings, zero, and false.

Operators	Required checks on RDF/SPARQL and property-graph/Cypher
Projection, distinct results, ordering, limits	Check selected variables, duplicates before and after distinct projection, ascending and descending ordering on declared values, and limits applied to the correct result scope.
Grouped COUNT, SUM, AVG, MIN, MAX	Test every function independently, then combine aggregation with joins, optional values, filters, projection, ordering, and limits. Check grouping keys, applicable datatypes, missing values, empty input, and multiplicity against model-specific expected results.

7.3 Benchmark protocol

The project selects the fixture size, hardware, network bound, repetition count and benchmark conditions below. The fixture contains 100,000 identifiable entities and 1,000,000 relationship facts, plus task attributes. Report RDF triple counts and property-graph node, relationship, and property counts separately; different encodings do not imply equal storage cost.

The reference client has at least four processor cores and 16 GB memory. Record actual hardware, client runtime, source hardware/software, indexes, cache state, and network condition. Local timing excludes network waits. End-to-end query tests use a controlled source with documented round-trip time at most 100 ms. Collect at least 30 repetitions per declared operation after documented warm-up. Separate cold/warm distributions and retain failures.

Measure search, categorical/range facets, single-hop expansion, aggregation, bounded paths, view switching, undo, cancellation, and reopening. Report each backend independently. A workload failing on one backend does not satisfy the baseline.

7.4 Task-based usability evaluation

Use the same information needs for both user groups: resolve ambiguity, restrict a set, explain a path, construct a query, and preserve a finding. Define initial state, correct answers, stopping criteria, and unaided completion. Record completion, wrong turns, actions, time, and participants' explanation of results.

The project's formative evaluation targets at least 80% unaided completion of predefined core tasks by non-experts. This is a target, not a study result. Report user groups and backends separately with participant experience. Settle recruitment, consent, and retention before collecting participant data. A small formative study does not establish population-wide effectiveness.

8 Traceability and release criteria

Use the individual origin statements for clause-level provenance. Table 6 summarises the mappings; its grouped references do not support every clause or demonstrate implementation or achieved targets.

Table 6: Requirements, sources, design considerations, and tests.

Requirements	Sources / consideration	Required response	Verification
FR-01-02	E04, E08-E11 / G06	Dual read-only connections and preserved semantics.	AC-01,07,08; NFR-01,06,08.
FR-03-04	E02, E03, E04 / G01	Search, disambiguation, facets.	AC-02; NFR-01,02,04.
FR-05-06	E03, E05, E08 / G02	Navigation, bounded paths, context.	AC-03,09; NFR-01,03-05.

Requirements	Sources / consideration	Required response	Verification
FR-07–08	E01, E04, E10–E11 / G01,G06	Visual profile and preserved native mode.	AC-04,08; NFR-01,06,08.
FR-09–10	E04, E07 / G03	Grounded suggestions and readable state.	AC-05; NFR-01,02.
FR-11–12	E01, E06, E08, E12 / G02,G05	Coordinated views and aggregation.	AC-06,09; NFR-01–05.
FR-13–15	E01, E04–E06, E08–E09, E11 / G04	Saved context and faithful exports.	AC-07; NFR-01,06,07,09.
FR-16	E01, E04 / G02,G06	Cancellation and recovery.	AC-08; NFR-05–07.
FR-12a,17	E01, E02, E04, E06 + project choices	Typed views and optional evaluation records.	Type/recording-control inspection; NFR-06,09.

Release requires every Must requirement and scenario to pass separately for RDF/SPARQL and property-graph/Cypher, including model-specific checks. A combined percentage shall not conceal a connector failure. Performance targets apply to declared conditions. An adopted target change updates the specification before release evaluation.

Future federation, additional views or query constructs, and collaboration must preserve the version 1 baseline. Architecture and stack choices must preserve the specified semantics and pass the behavioural tests.

References

- Berners-Lee, T., Chen, Y., Chilton, L., Connolly, D., Dhanaraj, R., Hollenbach, J., Lerer, A., & Sheets, D. (2006). Tabulator: Exploring and analyzing linked data on the Semantic Web. *Proceedings of the 3rd International Semantic Web User Interaction Workshop*. https://www.cs.columbia.edu/~chilton/web/my_publications/BernersLeeTabulator2006.pdf
- Ferré, S. (2017). Sparklis: An expressive query builder for SPARQL endpoints with guidance in natural language. *Semantic Web*, 8(3), 405–418. <https://doi.org/10.3233/SW-150208>
- Harris, S., & Seaborne, A. (Eds.). (2013). *SPARQL 1.1 query language*. World Wide Web Consortium. <https://www.w3.org/TR/2013/REC-sparql11-query-20130321/>
- Heim, P., Hellmann, S., Lehmann, J., Lohmann, S., & Stegemann, T. (2009). RelFinder: Revealing relationships in RDF knowledge bases. In *Semantic multimedia* (pp. 182–187). Springer. https://doi.org/10.1007/978-3-642-10543-2_21
- Hildebrand, M., van Ossenbruggen, J., & Hardman, L. (2006). /facet: A browser for heterogeneous Semantic Web repositories. In *The Semantic Web – ISWC 2006* (pp. 272–285). Springer. https://doi.org/10.1007/11926078_20
- Ikkala, E., Hyvönen, E., Rantala, H., & Koho, M. (2022). Sampo-UI: A full stack JavaScript framework for developing semantic portal user interfaces. *Semantic Web*, 13(1), 69–84. <https://doi.org/10.3233/SW-210428>
- Liebig, T., Opitz, M., Vialard, V., & Wenzel, M. (2023). Scalable no-code knowledge graph exploration and querying with SemSpect. In *Posters and Demo Track of the 19th International Conference on Semantic Systems* (CEUR Workshop Proceedings, Vol. 3526). CEUR-WS.org. <https://ceur-ws.org/Vol-3526/paper-07.pdf>
- Lissandrini, M., Mottin, D., Palpanas, T., & Velegrakis, Y. (2020). Graph-query suggestions for knowledge graph exploration. In *Proceedings of The Web Conference 2020* (pp. 2549–2555). Association for Computing Machinery. <https://doi.org/10.1145/3366423.3380005>
- Micsik, A., Tóth, Z., & Turbucz, S. (2014). LODmilla: Shared visualization of Linked Open Data.

- In *Theory and practice of digital libraries – TPDL 2013 selected workshops* (pp. 89–100). Springer. https://doi.org/10.1007/978-3-319-08425-1_9
- openCypher. (2026). *The openCypher property graph query language* (Release 2024.3) [Language specification and technology compatibility kit]. GitHub. <https://github.com/opencypher/openCypher/releases/tag/2024.3>
- World Wide Web Consortium. (2014). *RDF 1.1 concepts and abstract syntax*. <https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/>
- World Wide Web Consortium. (2024). *Web Content Accessibility Guidelines (WCAG) 2.2*. <https://www.w3.org/TR/2024/REC-WCAG22-20241212/>

A Local primary-source identifiers

ID	Local source
E01	<code>papers/2006_Berners-Lee_Tabulator_Exploring.pdf</code>
E02	<code>papers/2006_Hildebrand_facet_A.pdf</code>
E03	<code>papers/2009_Heim_RelFinder_Revealing.pdf</code>
E04	<code>papers/2017_Ferre_Sparklis_An.pdf</code>
E05	<code>papers/2014_Micsik_LODmilla_Shared.pdf</code>
E06	<code>papers/2022_Ikkala_Sampo-UI_A.pdf</code>
E07	<code>papers/2020_Lissandrini_Graph-Query_Suggestions.pdf</code>
E08	<code>papers/2023_Liebig_Scalable_No-Code.pdf</code>

B Glossary

Entity

A source-identified object inspected in the interface.

Relationship

An RDF predicate assertion or property-graph relationship with model-specific identity and context.

Facet

A displayed dimension restricting the active result set.

Workspace

Saved source references, query, constraints, display state, observations, and annotations.

Exact result

A declared query scope evaluated without sampling or an undisclosed limit.

Bounded result

Data restricted by an explicit count, depth, timeout, page, or sampling policy.

Evidence bundle

A finding bound to source, query, parameters, retrieval time, scope, limits, and annotations.

Paired acceptance

The same supported information need tested on both graph families, with separate model-specific checks.